Claude Code in Action

Tool Use · Context · Thinking · Hooks · MCP · SDK

mostly Opus 4.6

Course Overview

What is Claude Code?

- Coding assistants
- Tool use mechanism
- Claude Code demo

Getting Hands On

- Setup, context, permissions
- Making changes
- Thinking & effort levels
- Commands & shortcuts
- MCP servers (+ Context7)
- GitHub integration

Hooks, SDK & Tips

- Hook architecture
- PreToolUse / PostToolUse
- Implementing hooks
- Claude Agent SDK
- Workflow tips & pitfalls

What Is a Coding Assistant?

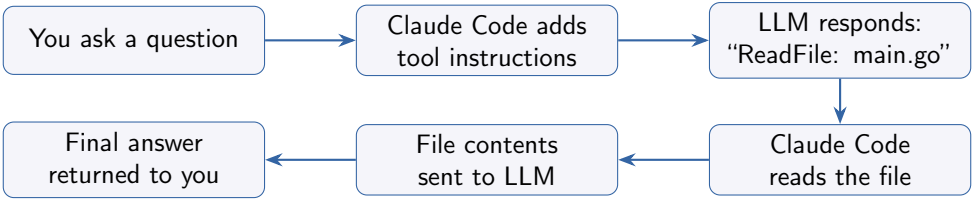
A coding assistant uses language models to tackle complex programming tasks.

How it works (same as a human developer):

1. **Gather context** — read files, understand errors, find relevant code
2. **Formulate a plan** — decide on a fix or implementation strategy
3. **Take action** — edit files, run commands, verify results

Steps 1 and 3 require the model to *interact with the outside world*.
But language models can only process text and return text...

The Tool Use Mechanism



Why Claude's tool use matters

Strong tool use ⇒ tackles harder tasks, extensible platform, better security (no full-codebase indexing).

Setup & Project Initialization

Installation:

```
npm install -g @anthropic-ai/claude-code
cd your-project
claude # launches Claude Code
```

The `/init` command:

- Analyzes your entire codebase
- Creates a `CLAUDE.md` summary (architecture, commands, patterns)
- Included in *every* request — persistent system prompt

Tip

Press **Shift+Tab** to cycle permission modes (e.g. auto-accept edits for the session).

Permissions

Three tool approval tiers:

Tool type	Example	Approval
Read-only	File reads, Grep	Automatic
Bash commands	Shell execution	Per command
File modification	Edit, Write	Per session

Permission modes (cycle with **Shift+Tab**):

- `default` — ask each tool
- `acceptEdits` — auto-approve edits
- `plan` — read-only
- `auto` — smart auto-approve
- `bypass` — skip all prompts

Pre-approve in `settings.json`:

```
"permissions": { "allow": ["Bash(npm run *)"],
                 "deny": ["Read(.env)"] }
```

Evaluation order: **deny** > ask > allow (deny always wins).

Adding Context: CLAUDE.md

Three locations, three scopes:

File	Scope	Shared?
`CLAUDE.md`	Project	Yes (committed)
`CLAUDE.local.md`	Project	No (personal)
`~/.claude/CLAUDE.md`	All projects	No (global)

Memory mode with #:

```
# Use comments sparingly. Only comment complex code.
```

Claude merges this instruction into your `CLAUDE.md` automatically.

File mentions with @:

```
How does the auth system work? @auth
```

Includes the file's contents in the request. Also works inside `CLAUDE.md`.

Making Changes

Key techniques:

- **Screenshots** — paste images directly; Claude reads error messages, UI issues
- **Planning mode** — one of the `Shift+Tab` modes; Claude outlines steps before executing
- **Adaptive thinking** — Claude adjusts reasoning depth; tune with `/effort`
- **Auto mode** — classifier auto-approves safe operations, blocks risky ones

Workflow

1. Describe the problem or feature
2. Let Claude explore, plan, and implement
3. Review the diff, approve or redirect

Thinking & Effort Levels

Adaptive Thinking (Opus 4.6 / Sonnet 4.6)

- Claude **dynamically allocates** reasoning depth
- Controlled via `/effort` command:

`low`	Fast, cheap — straightforward tasks
`medium`	Default balanced mode
`high`	Deep reasoning — complex problems
`max`	No token limit (Opus 4.6 only)

Magic keyword: include `ultrathink` anywhere in your prompt → sets effort to `high` for that one turn.

Controls

- `/effort high` — session-wide
- `-effort high` — at startup
- `Alt+T` — toggle thinking on/off
- `Ctrl+O` — show thinking output

Common mistake

"think harder" in your prompt does **nothing**. Only `ultrathink` and `/effort` actually change reasoning depth.

Controlling Context

Interrupt & Redirect

- **Escape** — stop mid-response, redirect
- **Escape** $\times$ **2** — rewind to earlier point
- Combine with # to fix repeating errors

Commands

- `/compact` — summarize history, keep knowledge
- `/clear` — fresh start, no prior context
- Auto-compaction at $\sim$95% context capacity

When to use what

`/compact` when switching to a *related* task (preserves context).
`/clear` when switching to an *unrelated* task (avoids confusion).

Custom Commands

Create reusable commands:

1. Create `.claude/commands/audit.md`
2. Filename becomes the command: `/audit`
3. Restart Claude Code

Commands with arguments (`$ARGUMENTS`):

```
Write comprehensive tests for: $ARGUMENTS

Testing conventions:
* Use Vitest with React Testing Library
* Place test files in __tests__/ next to source
* Name test files as [filename].test.ts(x)

Coverage: happy paths, edge cases, error states
```

Usage:

```
/write_tests the use-auth.ts file in the hooks directory
```

Handy Commands

Side questions

- `/btw <question>` — ask without polluting history
- Uses parent context, no tool access
- Single response, then dismissed
- Low cost (reuses prompt cache)

File references

- `@filename` — include file in context
- `@file.py#5-10` — specific line range
- `@folder/` — directory listing
- `Alt+K` (VS Code) — auto-insert selection

Diagnostics & Info

- `/doctor` — diagnose installation issues
- `/context` — visualize context usage
- `/cost` — token usage statistics
- `/diff` — interactive diff viewer

Session management

- `claude -n "name"` — name a session
- `claude -continue` — resume latest
- `/branch` — fork conversation
- `/export` — save as text file

MCP Servers

Model Context Protocol — extend Claude with new tools.

Example: Playwright (browser control)

```
claude mcp add playwright npx @playwright/mcp@latest
```

Pre-approve permissions in `.claude/settings.local.json`:

```
{
  "permissions": {
    "allow": ["mcp__playwright"],
    "deny": []
  }
}
```

Ecosystem: database interactions, API testing, cloud integrations, file operations, dev tool automation...

Deferred loading: MCP tools lazy-load via `ToolSearch`, keeping unused tool schemas out of context.

MCP in Practice: Context7

Problem: Claude's training data lags behind library releases → hallucinated APIs.
Solution: **Context7** MCP fetches *current* official documentation on demand.

Setup (one command):

```
claude mcp add context7 -- npx -y @upstash/context7-mcp@latest
```

Usage — just say "use context7":

```
> How does Next.js after() work? use context7
```

How it works:

1. `resolve-library-id` → finds the library
2. `query-docs` → fetches version-specific docs
3. Claude answers with *current* API

Why it matters:

- No hallucinated deprecated methods
- Working code on first try
- Version-specific examples
- Free, open-source (Upstash)

GitHub Integration

Run `/install-github-app` in Claude Code.

Two workflows added to `.github/workflows/`**:**

1. **Mention Action** — `@claude` in issues/PRs → Claude analyzes & responds
2. **PR Review Action** — auto-reviews every pull request

Customization options:

- `custom_instructions:` — project-specific context
- `mcp_config:` — attach MCP servers (e.g. Playwright)
- `allowed_tools:` — explicit tool permissions (required in CI)

Hooks: Architecture

PreToolUse

- Runs *before* tool call
- Can **block or rewrite** the operation
- Access control, validation

PostToolUse

- Runs *after* tool call
- Cannot block (already done)
- Formatting, testing, logging

Hook types: `"command"` (shell) | `"http"` (POST JSON to URL)
Config locations: `~/.claude/settings.json` (global) | `.claude/settings.json` (team) | `.claude/settings.local.json` (personal)

Hook Configuration

PreToolUse example — guard specific tool calls:

```
"PreToolUse": [{
  "matcher": "Read|Grep",
  "hooks": [{
    "type": "command",
    "command": "node ./hooks/read_hook.js"
  }]
}]
```

PostToolUse example — run formatter after edits:

```
"PostToolUse": [{
  "matcher": "Write|Edit|MultiEdit",
  "hooks": [{
    "type": "command",
    "command": "node ./hooks/format_hook.js"
  }]
}]
```

Implementing a Hook: Protect `.env`

```
async function main() {
  const chunks = [];
  for await (const chunk of process.stdin) {
    chunks.push(chunk);
  }
  const toolArgs = JSON.parse(
    Buffer.concat(chunks).toString()
  );

  const readPath =
    toolArgs.tool_input?.file_path
    || toolArgs.tool_input?.path || "";

  if (readPath.includes('.env')) {
    console.error("You cannot read the .env file");
    process.exit(2); // exit code 2 = block
  }
}

main();
```

Hook receives tool call data via `stdin` as JSON. Exit code 2 blocks the operation with an error message back to Claude.

Hook Gotchas & Practical Recipes

Common Gotchas

- Use **absolute paths** in hook commands
- Restart Claude Code after changing hooks
- Test hooks with simple cases first

Useful hook ideas:

- **TypeScript type-checking** — run `tsc` after edits, feed errors back
- **Auto-formatting** — run Prettier/Black after writes
- **Query duplication prevention** — detect duplicate SQL/API calls
- **Test runner** — auto-run tests on changed files
- **Sensitive file protection** — block access to credentials

Other hook events: Notification, Stop, SubagentStop, PreCompact, UserPromptSubmit, SessionStart, SessionEnd.

The Claude Code SDK

Claude Agent SDK — run Claude Code programmatically (TypeScript, Python, CLI).

```
import { query } from "@anthropic-ai/claude-code";

const prompt = "Look for duplicate queries in ./src/queries";

for await (const message of query({ prompt })) {
  console.log(JSON.stringify(message, null, 2));
}
```

Permissions: read-only by default. Enable writes with:

```
for await (const message of query({
  prompt,
  options: { allowedTools: ["Edit"] }
})) { /* ... */ }
```

Use cases: Git hooks for code review, CI/CD quality checks, automated documentation, build optimization scripts.

Workflow Tips

1. Plan before you code

- Use **plan mode** (`Shift+Tab`) for unfamiliar code
- Ask Claude to research first: "google how X works, then implement"
- For big features: have Claude interview you, write a spec, then start fresh

2. Keep memory fresh

- Periodically: "update your memory with what you learned"
- `/memory` to browse and prune stale entries
- Keep `CLAUDE.md` under 200 lines — specificity > length

3. Ask Claude to improve your workflow

- "What workflow improvements would you suggest for this project?"
- "Review my CLAUDE.md — what's redundant or missing?"
- `/insights` for session analytics

4. Manage context aggressively

- `/clear` between unrelated tasks
- `/compact` when switching related tasks
- Use subagents for long explorations
- `/btw` for quick side questions

Common Pitfalls

Kitchen sink sessions

Mixing unrelated tasks fills context with noise.
Fix: `/clear` between unrelated tasks.

Correcting the same bug 3×

Failed attempts accumulate, confusing Claude.
Fix: After 2 fails, `/clear` and write a better prompt.

Vague prompts

"fix the login bug" gives Claude nothing to work with.

Good habits

- Provide **verification**: tests, screenshots, expected output
- Use `@file#5-10` to point at specific code
- Name sessions: `claude -n "feature-auth"`
- Start in plan mode for unfamiliar codebases
- Paste images of errors — Claude reads screenshots
- Scope exploration: "investigate token refresh in `src/auth/` only"

Summary

Core Concepts

- Tool use makes LLMs practical
- `CLAUDE.md` = persistent context
- `@` mentions for file focus
- `/compact` vs `/clear`

Power Features

- Custom commands (`$ARGUMENTS`)
- MCP servers (deferred tool loading)
- Hooks (block, rewrite, HTTP)
- Claude Agent SDK
- Auto mode, adaptive thinking
- `/model` to switch models mid-session
- Voice (`/voice`), remote control

Questions?

Introduction to Agent Skills

SKILL.md · Frontmatter · Matching · Sharing

mostly Opus 4.6

Previously, on Claude Code in Action...

Tool Use

LLMs interact with the world through tool calls — read files, run commands, edit code.

MCP Servers

Model Context Protocol extends Claude with new tools. Deferred loading keeps context lean.

CLAUDE.md & Context

Persistent context file included in every request. `@` mentions for file focus. `/compact` vs. `/clear`.

Hooks & SDK

PreToolUse/PostToolUse hooks for validation and formatting. Claude Agent SDK for programmatic control.

Permissions

Three tiers (read/bash/edit). Five modes from `default` to `bypass`. Pre-approve in `settings.json`.

What Are Skills?

Skills = reusable markdown instructions that Claude automatically applies to the right tasks at the right time.

Key properties

- Each skill lives in a `SKILL.md` file with **name** and **description** in frontmatter
- Claude matches your request against skill descriptions — activates automatically
- Load *on demand* (unlike `CLAUDE.md` which loads every time)

Rule of thumb: if you find yourself explaining the same thing to Claude repeatedly, that's a skill waiting to be written.

Bundled skills: `/batch`, `/simplify`, `/debug`, `/claude-api` ship with Claude Code.

Skills vs. Other Features

Feature	When loaded	Activation	Scope
`CLAUDE.md`	Every conversation	Always	Project-wide
Slash commands	On invocation	Explicit (`/cmd`)	Project/personal
Skills	On demand	**Automatic** (semantic)	Project/personal
Hooks	On tool use	Trigger-based	Settings file
MCP servers	On tool use	Tool-based	Config file
Subagents	On delegation	Automatic	Agent definition

Skills activate when Claude *recognizes the situation* — no explicit invocation needed.

Where Skills Live

Personal Skills

- Location: `~/.claude/skills/`
- Follow you across *all* projects
- Your personal coding preferences

Project Skills

- Location: `.claude/skills/`
- Shared with anyone who clones the repo
- Team conventions & workflows

Priority hierarchy (name conflicts):

Enterprise → Personal → Project → Plugins

Creating Your First Skill

Directory structure:

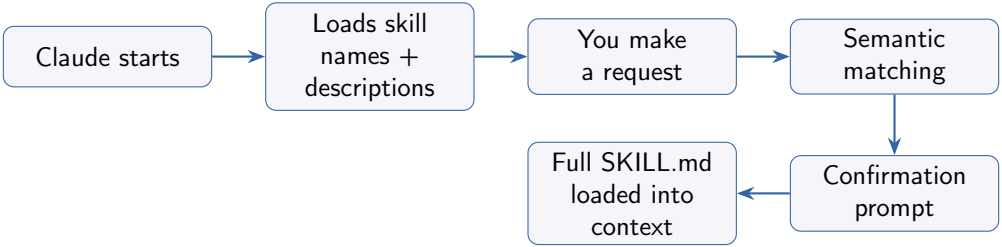
```
~/.claude/skills/
  pr-description/
    SKILL.md
```

SKILL.md with frontmatter:

```
---
name: pr-description
description: >
  Generate a pull request description from the
  current branch's changes. Summarize what changed,
  why, and any testing notes.
---

When asked to create a PR description:
1. Run `git diff main...HEAD` to see all changes
2. Summarize the purpose of the changes
3. List key files modified
4. Note any breaking changes or migration steps
5. Suggest reviewers based on file ownership
```

How Skill Matching Works



Important

Only names and descriptions are loaded at startup. The full skill body is loaded *only after confirmation* — keeps context window lean.

Configuration & Multi-File Skills

Frontmatter options:

```
---
name: deploy-checker
description: Validate deployment readiness
allowed-tools:
  - Bash
  - Read
model: sonnet
context: fork # run in isolated subagent
user-invocable: true # show in / menu
---
```

Progressive disclosure — multi-file skills:

```
.claude/skills/deploy-checker/
  SKILL.md # main entry point
  scripts/ # shell scripts, Python, etc.
  templates/ # report templates
```

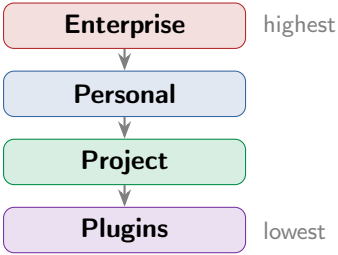
Skills can reference supporting files, scripts, and templates. Dynamic context injection available at load time.

Sharing Skills & Priority Hierarchy

How to share:

- **Git** — commit `.claude/skills/` to repo; team gets skills on clone
- **Plugins** — installable packages (growing directory), open standard (`agentskills.io`)
- **Enterprise** — org-wide deployment, enforce standards at scale
- **Composition** — skills can invoke subagents for powerful workflows

Priority (name conflicts):



Troubleshooting Skills

Common Issues

1. **Skill doesn't trigger** — description too vague; make it specific and action-oriented
2. **Wrong skill triggers** — descriptions overlap; differentiate with distinct verbs/contexts
3. **Skill not loading** — check file path, frontmatter YAML syntax, restart Claude
4. **Priority conflict** — remember: Enterprise > Personal > Project > Plugins
5. **Runtime errors** — test skill instructions manually first

Tip: Verify skill loading

Restart Claude Code and confirm your skill appears in the skill list. If it doesn't, check the file path and YAML frontmatter syntax.

Summary

Key Takeaways

- Skills = **reusable instructions** that activate automatically via semantic matching
- Frontmatter: `name`, `description`, `allowed-tools`, `model`, `context`, `user-invocable`
- Personal (`~/.claude/skills/`) vs. Project (`.claude/skills/`)
- Write specific descriptions — they are the matching key
- Share via git, plugins, or enterprise managed settings
- Multi-file skills with progressive disclosure and dynamic context injection

Questions?

Introduction to Subagents

Delegation · Custom Agents · Effective Design

mostly Opus 4.6

Previously, on Agent Skills...

What Are Skills?

Reusable markdown instructions (`SKILL.md`) that Claude activates automatically via semantic matching.

Frontmatter

`name`, `description`, `allowed-tools`, `model`, `context`, `user-invocable`.

Locations

Personal (`~/.claude/skills/`) vs. Project (`.claude/skills/`).

Matching

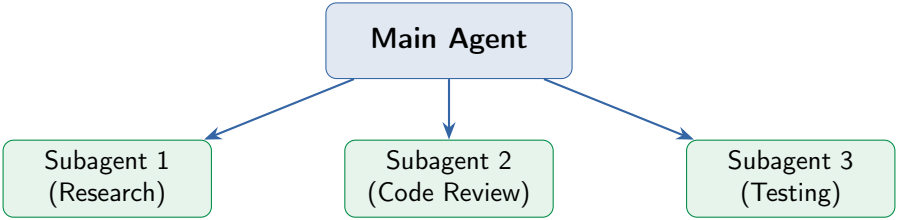
Only names + descriptions loaded at startup. Full body loaded after confirmation.

Sharing

Via git, plugins, or enterprise. Priority: Enterprise > Personal > Project > Plugins.

What Are Subagents?

Subagents = separate Claude instances with their own context window, spawned by the main agent to handle specific tasks.



- Each subagent has its own **separate context window**
- Can run **in parallel** for independent tasks
- Results flow back to the parent agent

Built-in Subagents

Claude Code ships with three built-in subagent types:

General Purpose

- Default agent type
- Full tool access
- Complex multi-step tasks

Explore

- Fast codebase search
- Find files by pattern
- Answer codebase questions

Plan

- Architecture design
- Implementation strategy
- Trade-off analysis

You can also create **custom subagents** tailored to your workflows.

Creating a Custom Subagent

Create a markdown file in `.claude/agents/`:

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
description: >
  Reviews code changes for security vulnerabilities,
  checks for OWASP top 10 issues, and suggests fixes.
tools:
  - Read
  - Grep
  - Glob
model: opus
isolation: worktree # isolated repo copy
maxTurns: 20
---

When reviewing code for security issues:
1. Check for injection vulnerabilities (SQL, XSS, command)
2. Verify input validation at system boundaries
3. Check authentication and authorization logic
4. Look for sensitive data exposure
5. Report findings with severity and fix suggestions
```

Designing Effective Subagents

The description is everything — it shapes how the parent delegates.

Bad description

"Helps with code"

Good description

"Reviews Python code changes for type safety issues, checks mypy compatibility, and suggests type annotations for untyped functions."

Design principles:

1. Specific, action-oriented description
2. Define **output format**
3. Instruct: report obstacles, don't guess
4. **Limit tool access** to what's needed
5. Consider **proactive delegation**

When Subagents Shine vs. When They Hurt

Use subagents for

- **Research** — explore codebase, gather context in parallel
- **Code reviews** — isolated analysis with focused tools
- **Custom system prompts** — specialized behavior per task
- **Parallel independent work**

Avoid subagents for

- **Expert claims** — "I'm an expert in X" doesn't help
- **Sequential pipelines** — overhead without parallelism benefit
- **Test runners** — use hooks instead for automatic testing
- **Simple single-step tasks**

The Decision Rule

Use a subagent when the task benefits from **isolation** (separate context) or **parallelism** (multiple tasks at once). Otherwise, keep it in the main agent.

Advanced Subagent Features

Background Execution

- Parent sets `run_in_background: true`
- Or **Ctrl+B** to background a running task
- Auto-denies unapproved permissions

Worktree Isolation

- `isolation: "worktree"` in frontmatter
- Subagent gets an isolated repo copy
- Enables safe parallel development

Shared Memory

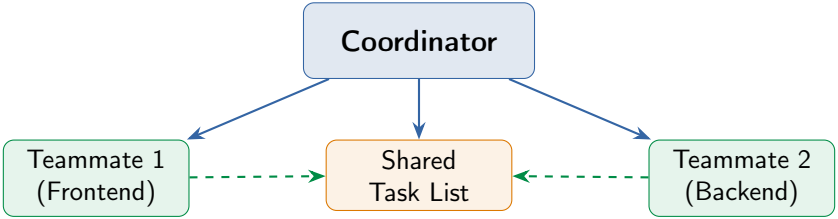
- Agents share the file-based memory system (`~/.claude/` directory)
- Knowledge accumulates across sessions

Invocation Methods

- Natural language (auto-delegation)
- `@"reviewer (agent)"` (explicit mention)
- SDK: `query({ agent: "reviewer" })`

Agent Teams (Research Preview)

Agent Teams = multiple Claude sessions coordinating across separate processes.



Key difference from subagents

Subagents work *within* a single session. Agent teams coordinate *across* separate sessions with shared task lists and mailbox systems.

Summary

Key Takeaways

- Subagents = separate Claude instances with their own context
- Built-in: General Purpose, Explore, Plan
- Custom subagents via `.claude/agents/*.md` with frontmatter
- Description quality determines delegation quality
- Advanced: background execution, worktree isolation, persistent memory
- Agent Teams = multi-session coordination (research preview)
- Best for: research, reviews, parallel work

Questions?

Introduction to Claude Cowork

Delegation · Plugins · Scheduling · Research

mostly Opus 4.6

Previously, on Subagents...

Subagents

Separate Claude instances with their own context window, spawned for specific tasks.

Built-in Types

General Purpose (full tools), Explore (fast search), Plan (architecture design).

Custom Agents

Define in `.claude/agents/*.md` with frontmatter: tools, model, isolation, maxTurns.

When to Use

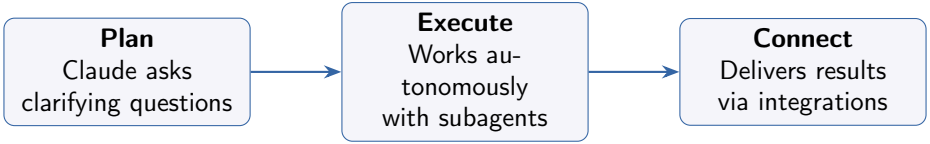
Research, code reviews, parallel work. Avoid for simple tasks or sequential pipelines.

Advanced Features

Background execution, worktree isolation, agent teams for multi-session coordination.

What Is Cowork?

Cowork = Claude's delegation mode in the desktop app.
Instead of conversation, you *hand off* a complete task and step away.



Three modes in Claude desktop:

- **Chat** — conversational back-and-forth
- **Cowork** — delegate and step away
- **Code** — Claude Code in the IDE

Platforms: macOS and Windows (since February 2026).

Getting Set Up

Requirements:

- Claude desktop app
- Folder access configured (grant access to working directories)
- Connectors enabled (Google Drive, Notion, Slack, etc.)
- Chrome extension (optional, for web tasks)

Key difference from Chat

In Cowork, you describe what you want done and *walk away*. Claude will ask clarifying questions only if needed, then execute autonomously.

The Task Loop

1. **Describe** your task clearly
2. **Questions** — Claude may ask for clarification
3. **Step away** — Claude works with subagents in parallel
4. **Open finished work** — review the deliverables

Subagents in Cowork: (see *Subagents* deck)

- Cowork spawns multiple subagents for parallel execution
- Each handles a piece of the overall task
- Results are merged into a coherent deliverable

Example

"Organize all PDFs in my Downloads folder by topic, rename them with a consistent naming scheme, and create a summary index."

Giving Cowork Context

Projects

- Upload reference files to a project
- Claude uses them as knowledge base
- Persistent across tasks

Instructions Panel

- Per-project custom instructions
- Tone, format, conventions

Global Instructions

- Apply to all Cowork tasks
- Your preferences everywhere

In the prompt itself

- Attach files, screenshots
- Reference URLs
- Be specific about desired output

Plugins: Skills + Connectors + Subagents

Plugins bundle capabilities into installable packages:

Skills	Connectors	Subagents
Reusable instructions (see *Agent Skills* deck)	Integrations with external services	Specialized agent definitions

Installation: Browse the plugin directory (growing catalog), install with one click.
Structure:

```
my-plugin/
  skills/
    data-analysis/SKILL.md
  connectors/
    sheets-connector.json
  agents/
    analyst.md
```

Scheduled Tasks & File Operations

Scheduled Tasks (`/schedule`):

- Set up recurring Cowork tasks
- Define cadence (daily, weekly, custom)
- Manage and monitor from the app

File & Document Tasks:

Follow the prompt pattern: **Input** → **Transformation** → **Output**

Examples

- "Take these 20 customer emails, extract feature requests, create a prioritized spreadsheet"
- "Read all markdown files in docs/, fix formatting, add missing headers"
- "Convert this PDF report into a slide deck with key findings"

Research & Analysis at Scale

Cowork excels at **volume, parallelism, and in-place computation**.

Tips for effective research prompts:

- Ask **sharp, specific questions** (not broad/vague)
- Specify the **output format** you want
- Provide **source materials** or point to directories
- Let Cowork parallelize across subagents

Permissions & Model Choice

- Cowork runs in **isolated execution** — controlled access
- Review outputs before acting on them
- Model choice: **Opus** (complex), **Sonnet** (balanced), **Haiku** (fast/cheap)

Computer Use

Claude can control your computer — open apps, navigate browsers, fill forms, click UI elements.

How it works:

1. Grant Claude screen and input access
2. Describe the task (e.g. "book a meeting with Alex on Tuesday")
3. Claude navigates the screen autonomously
4. Review and confirm actions

Use cases

- Fill out forms across multiple tools
- Navigate legacy systems without APIs
- Automate repetitive desktop workflows

Limitations

- macOS primary; Windows support expanding
- Review actions before confirming
- Not suitable for high-stakes inputs

Summary

Key Takeaways

- Cowork = **delegation mode**: describe task, step away, get results
- Task loop: Describe → Clarify → Execute → Deliver
- Context via: projects, instructions panel, global settings, inline
- Plugins bundle skills + connectors + subagents
- `/schedule` for recurring tasks
- File tasks: Input → Transformation → Output pattern
- Computer Use: desktop automation (macOS; review before confirming)
- Research: sharp questions, specified format, parallelized execution

Questions?

Claude 101

Prompting · Projects · Artifacts · Skills · Research

mostly Opus 4.6

Course Overview

Foundations

- What is Claude?
- First conversation
- Getting better results
- Desktop app modes

Features

- Projects
- Artifacts
- Skills
- Connectors & MCP
- Enterprise search
- Research mode

Application

- Use cases by role
- Other ways to work (Code, Slack, Excel)

What Is Claude?

Claude = AI assistant by Anthropic, designed to be **helpful, harmless, and honest**.

Built with Constitutional AI:

- Trained on a set of principles (a "constitution")
- Self-evaluates responses against those principles
- Aims for safe, balanced, truthful outputs

Key capabilities:

- Long-context understanding (Sonnet: 200K, Opus: 1M tokens)
- Strong reasoning and analysis
- Code generation and review
- Document creation and summarization
- Tool use and integration
- Adaptive thinking — adjusts reasoning depth to task complexity

Access: claude.ai, desktop app, API, Claude Code CLI, Slack, Excel...

Your First Conversation: Prompting

The 3-part prompt structure:

1. **Set the stage** — give Claude a role and context
2. **Define the task** — be specific about what you need
3. **Specify rules** — format, constraints, what to avoid

Example

"You are a senior data analyst. Analyze this CSV of quarterly sales data. Create a summary table showing trends by region. Use bullet points, not paragraphs. Flag any anomalies."

Uploads: paste images, attach files (PDFs, CSVs, code) directly in the conversation.
Iteration: Claude's first answer is a starting point — refine and redirect.

Getting Better Results

Common challenges:

- Too generic / vague output
- Wrong format or structure
- Missing key details
- Hallucinated information
- Over-verbose responses

AI Fluency 4D Framework:

1. **Discover** — explore what's possible
2. **Define** — scope the task precisely
3. **Develop** — iterate on outputs
4. **Deliver** — finalize and apply

Tips for better prompts:

- Be specific, not vague
- Provide examples of desired output
- Break complex tasks into steps
- Specify format and length
- Give context about your audience
- Use **evals** to measure quality systematically

Desktop App: Chat, Cowork, Code

	Chat	**Cowork**	**Code**
Interaction	Conversational	Delegation	CLI/IDE
Best for	Q&A, drafting	Complex tasks	Software dev
Autonomy	Low	High	High
File access	Upload	Folder access	Full repo
Subagents	No	Yes	Yes

Quick entry: launch from system tray, dictation support.

Choose the right mode

Chat for quick questions. **Cowork** for "do this for me" tasks. **Code** for development.

Projects

Projects = persistent knowledge bases for Claude.

What you can add:

- Reference documents (PDFs, docs)
- Custom instructions
- Style guides and templates
- Data files

How it works:

- RAG (Retrieval Augmented Generation)
- Claude searches project files for relevant context
- No need to re-upload every conversation

Collaboration:

- Share projects with team members
- Everyone gets the same context
- Consistent instructions across users

Use cases

- Company style guide project
- Product documentation hub
- Research paper collection
- Client-specific knowledge base

Artifacts

Artifacts = rich, interactive outputs Claude creates alongside responses.

Types of artifacts:

- **Documents** — formatted text, reports
- **Code** — syntax-highlighted, runnable
- **HTML/CSS** — rendered web pages
- **SVG** — vector graphics
- **Mermaid** — diagrams and flowcharts
- **React components** — interactive UIs
- **PowerPoint** — downloadable `.pptx` files
- Shareable, publishable, MCP-connected

Workflow

Ask Claude to create → preview in the artifact panel → iterate → share or publish.

Skills & Connectors

Skills

- **Anthropic skills** — built-in capabilities
- **Custom skills** — user/team-defined (see *Agent Skills* deck)
- Enable from settings
- Activate automatically based on context

Skills vs. Projects:
Skills = *how* to do things
Projects = *what* to know

Connectors

- Google Drive, Notion, Slack...
- MCP (Model Context Protocol)
- Web connectors for any service
- Desktop extensions

Enterprise Search:

- Admin configures data sources
- Users query across all connected systems
- Security: respects original permissions

Research Mode

Research mode = multi-step investigation with extended thinking.

How it works:

1. Claude breaks your question into sub-questions
2. Searches across available sources
3. Synthesizes findings with **citations**
4. Produces a structured research report

Best for:

- Complex questions requiring multiple sources (up to 45 min)
- Competitive analysis and market research
- Literature review and synthesis
- Due diligence and fact-checking

Tip

Research mode uses extended thinking — takes longer but produces deeper, more accurate results.

Use Cases by Role

- **Sales** — prospect research, email drafting, proposal generation
- **Marketing** — content creation, campaign analysis, copywriting
- **Finance** — data analysis, report generation, modeling assistance
- **HR** — policy drafting, interview prep, training materials
- **Legal** — document review, contract analysis, compliance checks
- **Research** — literature synthesis, experiment design, data interpretation

Other interfaces: Claude Code (CLI) · Slack integration · Excel add-in · Chrome extension

Summary

Key Takeaways

- Claude: helpful, harmless, honest — Constitutional AI, adaptive thinking
- Prompt = set stage + define task + specify rules; iterate
- Three modes: Chat (quick), Cowork (delegate), Code (dev)
- Projects = persistent knowledge; Artifacts = rich outputs
- Skills activate automatically; Connectors extend reach
- Research mode for deep multi-source investigation
- Available everywhere: web, desktop, CLI, API, Slack, Excel

Questions?